

well for the subproblem with keys  $k_i, \dots, k_j$  and dummy keys  $d_{i-1}, \dots, d_j$ . The usual cut-and-paste argument applies. If there were a subtree  $T''$  whose expected cost is lower than that of  $T'$ , then we could cut  $T'$  out of  $T$  and paste in  $T''$ , resulting in a binary search tree of lower expected cost than  $T$ , thus contradicting the optimality of  $T$ .

We need to use the optimal substructure to show that we can construct an optimal solution to the problem from optimal solutions to subproblems. Given keys  $k_i, \dots, k_j$ , one of these keys, say  $k_r$  ( $i \leq r \leq j$ ), is the root of an optimal subtree containing these keys. The left subtree of the root  $k_r$  contains the keys  $k_i, \dots, k_{r-1}$  (and dummy keys  $d_{i-1}, \dots, d_{r-1}$ ), and the right subtree contains the keys  $k_{r+1}, \dots, k_j$  (and dummy keys  $d_r, \dots, d_j$ ). As long as we examine all candidate roots  $k_r$ , where  $i \leq r \leq j$ , and we determine all optimal binary search trees containing  $k_i, \dots, k_{r-1}$  and those containing  $k_{r+1}, \dots, k_j$ , we are guaranteed that we will find an optimal binary search tree.

There is one detail worth noting about “empty” subtrees. Suppose that in a subtree with keys  $k_i, \dots, k_j$ , we select  $k_i$  as the root. By the above argument,  $k_i$ ’s left subtree contains the keys  $k_i, \dots, k_{i-1}$ . We interpret this sequence as containing no keys. Bear in mind, however, that subtrees also contain dummy keys. We adopt the convention that a subtree containing keys  $k_i, \dots, k_{i-1}$  has no actual keys but does contain the single dummy key  $d_{i-1}$ . Symmetrically, if we select  $k_j$  as the root, then  $k_j$ ’s right subtree contains the keys  $k_{j+1}, \dots, k_j$ ; this right subtree contains no actual keys, but it does contain the dummy key  $d_j$ .

## Step 2: A recursive solution

We are ready to define the value of an optimal solution recursively. We pick our subproblem domain as finding an optimal binary search tree containing the keys  $k_i, \dots, k_j$ , where  $i \geq 1$ ,  $j \leq n$ , and  $j \geq i - 1$ . (When  $j = i - 1$ , there are no actual keys; we have just the dummy key  $d_{i-1}$ .) Let us define  $e[i, j]$  as the expected cost of searching an optimal binary search tree containing the keys  $k_i, \dots, k_j$ . Ultimately, we wish to compute  $e[1, n]$ .

The easy case occurs when  $j = i - 1$ . Then we have just the dummy key  $d_{i-1}$ . The expected search cost is  $e[i, i - 1] = q_{i-1}$ .

When  $j \geq i$ , we need to select a root  $k_r$  from among  $k_i, \dots, k_j$  and then make an optimal binary search tree with keys  $k_i, \dots, k_{r-1}$  as its left subtree and an optimal binary search tree with keys  $k_{r+1}, \dots, k_j$  as its right subtree. What happens to the expected search cost of a subtree when it becomes a subtree of a node? The depth of each node in the subtree increases by 1. By equation (15.11), the expected search cost of this subtree increases by the sum of all the probabilities in the subtree. For a subtree with keys  $k_i, \dots, k_j$ , let us denote this sum of probabilities as

$$w(i, j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l . \quad (15.12)$$

Thus, if  $k_r$  is the root of an optimal subtree containing keys  $k_i, \dots, k_j$ , we have

$$e[i, j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j)) .$$

Noting that

$$w(i, j) = w(i, r-1) + p_r + w(r+1, j) ,$$

we rewrite  $e[i, j]$  as

$$e[i, j] = e[i, r-1] + e[r+1, j] + w(i, j) . \quad (15.13)$$

The recursive equation (15.13) assumes that we know which node  $k_r$  to use as the root. We choose the root that gives the lowest expected search cost, giving us our final recursive formulation:

$$e[i, j] = \begin{cases} q_{i-1} & \text{if } j = i - 1 , \\ \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & \text{if } i \leq j . \end{cases} \quad (15.14)$$

The  $e[i, j]$  values give the expected search costs in optimal binary search trees. To help us keep track of the structure of optimal binary search trees, we define  $root[i, j]$ , for  $1 \leq i \leq j \leq n$ , to be the index  $r$  for which  $k_r$  is the root of an optimal binary search tree containing keys  $k_i, \dots, k_j$ . Although we will see how to compute the values of  $root[i, j]$ , we leave the construction of an optimal binary search tree from these values as Exercise 15.5-1.

### Step 3: Computing the expected search cost of an optimal binary search tree

At this point, you may have noticed some similarities between our characterizations of optimal binary search trees and matrix-chain multiplication. For both problem domains, our subproblems consist of contiguous index subranges. A direct, recursive implementation of equation (15.14) would be as inefficient as a direct, recursive matrix-chain multiplication algorithm. Instead, we store the  $e[i, j]$  values in a table  $e[1 \dots n+1, 0 \dots n]$ . The first index needs to run to  $n+1$  rather than  $n$  because in order to have a subtree containing only the dummy key  $d_n$ , we need to compute and store  $e[n+1, n]$ . The second index needs to start from 0 because in order to have a subtree containing only the dummy key  $d_0$ , we need to compute and store  $e[1, 0]$ . We use only the entries  $e[i, j]$  for which  $j \geq i-1$ . We also use a table  $root[i, j]$ , for recording the root of the subtree containing keys  $k_i, \dots, k_j$ . This table uses only the entries for which  $1 \leq i \leq j \leq n$ .

We will need one other table for efficiency. Rather than compute the value of  $w(i, j)$  from scratch every time we are computing  $e[i, j]$ —which would take

$\Theta(j - i)$  additions—we store these values in a table  $w[1 \dots n + 1, 0 \dots n]$ . For the base case, we compute  $w[i, i - 1] = q_{i-1}$  for  $1 \leq i \leq n + 1$ . For  $j \geq i$ , we compute

$$w[i, j] = w[i, j - 1] + p_j + q_j. \quad (15.15)$$

Thus, we can compute the  $\Theta(n^2)$  values of  $w[i, j]$  in  $\Theta(1)$  time each.

The pseudocode that follows takes as inputs the probabilities  $p_1, \dots, p_n$  and  $q_0, \dots, q_n$  and the size  $n$ , and it returns the tables  $e$  and  $root$ .

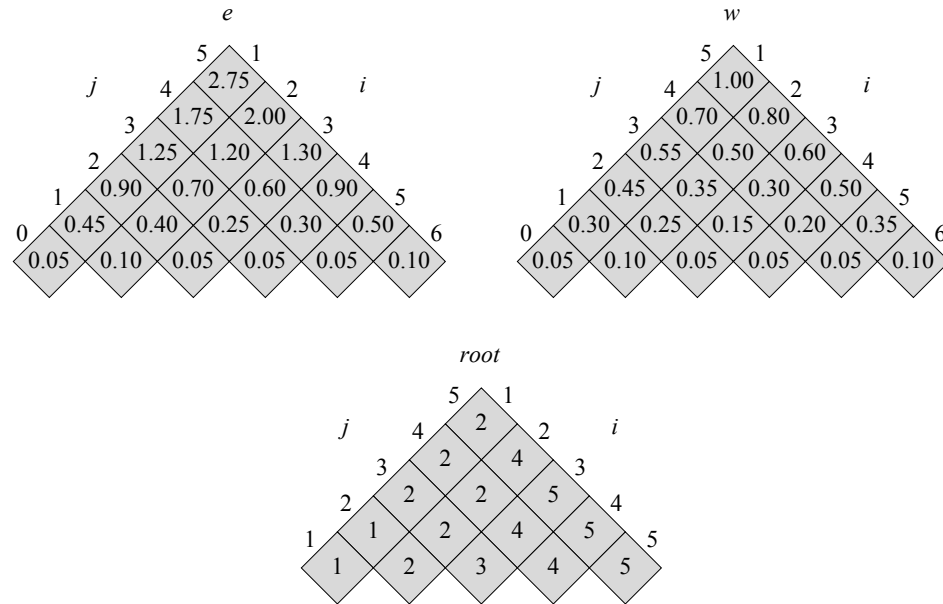
OPTIMAL-BST( $p, q, n$ )

```

1  let  $e[1 \dots n + 1, 0 \dots n]$ ,  $w[1 \dots n + 1, 0 \dots n]$ ,
   and  $root[1 \dots n, 1 \dots n]$  be new tables
2  for  $i = 1$  to  $n + 1$ 
3       $e[i, i - 1] = q_{i-1}$ 
4       $w[i, i - 1] = q_{i-1}$ 
5  for  $l = 1$  to  $n$ 
6      for  $i = 1$  to  $n - l + 1$ 
7           $j = i + l - 1$ 
8           $e[i, j] = \infty$ 
9           $w[i, j] = w[i, j - 1] + p_j + q_j$ 
10         for  $r = i$  to  $j$ 
11              $t = e[i, r - 1] + e[r + 1, j] + w[i, j]$ 
12             if  $t < e[i, j]$ 
13                  $e[i, j] = t$ 
14                  $root[i, j] = r$ 
15  return  $e$  and  $root$ 
```

From the description above and the similarity to the MATRIX-CHAIN-ORDER procedure in Section 15.2, you should find the operation of this procedure to be fairly straightforward. The **for** loop of lines 2–4 initializes the values of  $e[i, i - 1]$  and  $w[i, i - 1]$ . The **for** loop of lines 5–14 then uses the recurrences (15.14) and (15.15) to compute  $e[i, j]$  and  $w[i, j]$  for all  $1 \leq i \leq j \leq n$ . In the first iteration, when  $l = 1$ , the loop computes  $e[i, i]$  and  $w[i, i]$  for  $i = 1, 2, \dots, n$ . The second iteration, with  $l = 2$ , computes  $e[i, i + 1]$  and  $w[i, i + 1]$  for  $i = 1, 2, \dots, n - 1$ , and so forth. The innermost **for** loop, in lines 10–14, tries each candidate index  $r$  to determine which key  $k_r$  to use as the root of an optimal binary search tree containing keys  $k_i, \dots, k_j$ . This **for** loop saves the current value of the index  $r$  in  $root[i, j]$  whenever it finds a better key to use as the root.

Figure 15.10 shows the tables  $e[i, j]$ ,  $w[i, j]$ , and  $root[i, j]$  computed by the procedure OPTIMAL-BST on the key distribution shown in Figure 15.9. As in the matrix-chain multiplication example of Figure 15.5, the tables are rotated to make



**Figure 15.10** The tables  $e[i, j]$ ,  $w[i, j]$ , and  $root[i, j]$  computed by OPTIMAL-BST on the key distribution shown in Figure 15.9. The tables are rotated so that the diagonals run horizontally.

the diagonals run horizontally. OPTIMAL-BST computes the rows from bottom to top and from left to right within each row.

The OPTIMAL-BST procedure takes  $\Theta(n^3)$  time, just like MATRIX-CHAIN-ORDER. We can easily see that its running time is  $O(n^3)$ , since its **for** loops are nested three deep and each loop index takes on at most  $n$  values. The loop indices in OPTIMAL-BST do not have exactly the same bounds as those in MATRIX-CHAIN-ORDER, but they are within at most 1 in all directions. Thus, like MATRIX-CHAIN-ORDER, the OPTIMAL-BST procedure takes  $\Omega(n^3)$  time.

## Exercises

### 15.5-1

Write pseudocode for the procedure CONSTRUCT-OPTIMAL-BST( $root$ ) which, given the table  $root$ , outputs the structure of an optimal binary search tree. For the example in Figure 15.10, your procedure should print out the structure

$k_2$  is the root  
 $k_1$  is the left child of  $k_2$   
 $d_0$  is the left child of  $k_1$   
 $d_1$  is the right child of  $k_1$   
 $k_5$  is the right child of  $k_2$   
 $k_4$  is the left child of  $k_5$   
 $k_3$  is the left child of  $k_4$   
 $d_2$  is the left child of  $k_3$   
 $d_3$  is the right child of  $k_3$   
 $d_4$  is the right child of  $k_4$   
 $d_5$  is the right child of  $k_5$

corresponding to the optimal binary search tree shown in Figure 15.9(b).

### 15.5-2

Determine the cost and structure of an optimal binary search tree for a set of  $n = 7$  keys with the following probabilities:

| $i$   | 0    | 1    | 2    | 3    | 4    | 5    | 6    | 7    |
|-------|------|------|------|------|------|------|------|------|
| $p_i$ |      | 0.04 | 0.06 | 0.08 | 0.02 | 0.10 | 0.12 | 0.14 |
| $q_i$ | 0.06 | 0.06 | 0.06 | 0.06 | 0.05 | 0.05 | 0.05 | 0.05 |

### 15.5-3

Suppose that instead of maintaining the table  $w[i, j]$ , we computed the value of  $w(i, j)$  directly from equation (15.12) in line 9 of OPTIMAL-BST and used this computed value in line 11. How would this change affect the asymptotic running time of OPTIMAL-BST?

### 15.5-4 ★

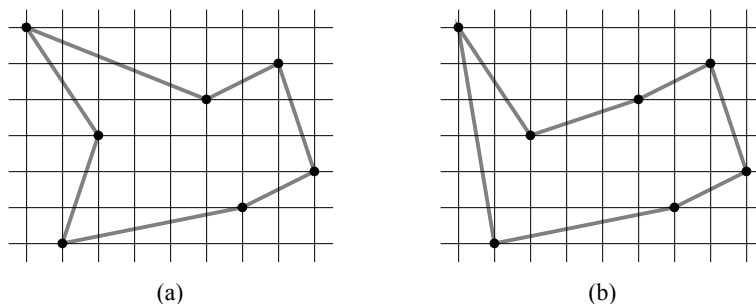
Knuth [212] has shown that there are always roots of optimal subtrees such that  $root[i, j - 1] \leq root[i, j] \leq root[i + 1, j]$  for all  $1 \leq i < j \leq n$ . Use this fact to modify the OPTIMAL-BST procedure to run in  $\Theta(n^2)$  time.

---

## Problems

### 15-1 Longest simple path in a directed acyclic graph

Suppose that we are given a directed acyclic graph  $G = (V, E)$  with real-valued edge weights and two distinguished vertices  $s$  and  $t$ . Describe a dynamic-programming approach for finding a longest weighted simple path from  $s$  to  $t$ . What does the subproblem graph look like? What is the efficiency of your algorithm?



**Figure 15.11** Seven points in the plane, shown on a unit grid. **(a)** The shortest closed tour, with length approximately 24.89. This tour is not bitonic. **(b)** The shortest bitonic tour for the same set of points. Its length is approximately 25.58.

### 15-2 Longest palindrome subsequence

A *palindrome* is a nonempty string over some alphabet that reads the same forward and backward. Examples of palindromes are all strings of length 1, `civic`, `racecar`, and `aibohphobia` (fear of palindromes).

Give an efficient algorithm to find the longest palindrome that is a subsequence of a given input string. For example, given the input `character`, your algorithm should return `carac`. What is the running time of your algorithm?

### 15-3 Bitonic euclidean traveling-salesman problem

In the *euclidean traveling-salesman problem*, we are given a set of  $n$  points in the plane, and we wish to find the shortest closed tour that connects all  $n$  points. Figure 15.11(a) shows the solution to a 7-point problem. The general problem is NP-hard, and its solution is therefore believed to require more than polynomial time (see Chapter 34).

J. L. Bentley has suggested that we simplify the problem by restricting our attention to *bitonic tours*, that is, tours that start at the leftmost point, go strictly rightward to the rightmost point, and then go strictly leftward back to the starting point. Figure 15.11(b) shows the shortest bitonic tour of the same 7 points. In this case, a polynomial-time algorithm is possible.

Describe an  $O(n^2)$ -time algorithm for determining an optimal bitonic tour. You may assume that no two points have the same  $x$ -coordinate and that all operations on real numbers take unit time. (*Hint*: Scan left to right, maintaining optimal possibilities for the two parts of the tour.)

### 15-4 Printing neatly

Consider the problem of neatly printing a paragraph with a monospaced font (all characters having the same width) on a printer. The input text is a sequence of  $n$

words of lengths  $l_1, l_2, \dots, l_n$ , measured in characters. We want to print this paragraph neatly on a number of lines that hold a maximum of  $M$  characters each. Our criterion of “neatness” is as follows. If a given line contains words  $i$  through  $j$ , where  $i \leq j$ , and we leave exactly one space between words, the number of extra space characters at the end of the line is  $M - j + i - \sum_{k=i}^j l_k$ , which must be nonnegative so that the words fit on the line. We wish to minimize the sum, over all lines except the last, of the cubes of the numbers of extra space characters at the ends of lines. Give a dynamic-programming algorithm to print a paragraph of  $n$  words neatly on a printer. Analyze the running time and space requirements of your algorithm.

### 15-5 Edit distance

In order to transform one source string of text  $x[1..m]$  to a target string  $y[1..n]$ , we can perform various transformation operations. Our goal is, given  $x$  and  $y$ , to produce a series of transformations that change  $x$  to  $y$ . We use an array  $z$ —assumed to be large enough to hold all the characters it will need—to hold the intermediate results. Initially,  $z$  is empty, and at termination, we should have  $z[j] = y[j]$  for  $j = 1, 2, \dots, n$ . We maintain current indices  $i$  into  $x$  and  $j$  into  $z$ , and the operations are allowed to alter  $z$  and these indices. Initially,  $i = j = 1$ . We are required to examine every character in  $x$  during the transformation, which means that at the end of the sequence of transformation operations, we must have  $i = m + 1$ .

We may choose from among six transformation operations:

**Copy** a character from  $x$  to  $z$  by setting  $z[j] = x[i]$  and then incrementing both  $i$  and  $j$ . This operation examines  $x[i]$ .

**Replace** a character from  $x$  by another character  $c$ , by setting  $z[j] = c$ , and then incrementing both  $i$  and  $j$ . This operation examines  $x[i]$ .

**Delete** a character from  $x$  by incrementing  $i$  but leaving  $j$  alone. This operation examines  $x[i]$ .

**Insert** the character  $c$  into  $z$  by setting  $z[j] = c$  and then incrementing  $j$ , but leaving  $i$  alone. This operation examines no characters of  $x$ .

**Twiddle** (i.e., exchange) the next two characters by copying them from  $x$  to  $z$  but in the opposite order; we do so by setting  $z[j] = x[i + 1]$  and  $z[j + 1] = x[i]$  and then setting  $i = i + 2$  and  $j = j + 2$ . This operation examines  $x[i]$  and  $x[i + 1]$ .

**Kill** the remainder of  $x$  by setting  $i = m + 1$ . This operation examines all characters in  $x$  that have not yet been examined. This operation, if performed, must be the final operation.

As an example, one way to transform the source string `algorithm` to the target string `altruistic` is to use the following sequence of operations, where the underlined characters are  $x[i]$  and  $z[j]$  after the operation:

| Operation              | $x$                 | $z$                 |
|------------------------|---------------------|---------------------|
| <i>initial strings</i> | <u>a</u> lgorithm   | —                   |
| copy                   | a <u>l</u> gorithm  | a_                  |
| copy                   | al <u>g</u> orithm  | al_                 |
| replace by t           | alg <u>o</u> rithm  | alt_                |
| delete                 | algor <u>i</u> thm  | alt_                |
| copy                   | algori <u>t</u> hm  | altr_               |
| insert u               | algori <u>u</u> thm | altru_              |
| insert i               | algori <u>i</u> thm | altrui_             |
| insert s               | algori <u>s</u> thm | altruiss_           |
| twiddle                | algorith <u>m</u>   | altruist <u>i</u> _ |
| insert c               | algorith <u>m</u>   | altruistic_         |
| kill                   | algorithm_          | altruistic_         |

Note that there are several other sequences of transformation operations that transform `algorithm` to `altruistic`.

Each of the transformation operations has an associated cost. The cost of an operation depends on the specific application, but we assume that each operation's cost is a constant that is known to us. We also assume that the individual costs of the copy and replace operations are less than the combined costs of the delete and insert operations; otherwise, the copy and replace operations would not be used. The cost of a given sequence of transformation operations is the sum of the costs of the individual operations in the sequence. For the sequence above, the cost of transforming `algorithm` to `altruistic` is

$$(3 \cdot \text{cost}(\text{copy})) + \text{cost}(\text{replace}) + \text{cost}(\text{delete}) + (4 \cdot \text{cost}(\text{insert})) \\ + \text{cost}(\text{twiddle}) + \text{cost}(\text{kill}) .$$

- a.* Given two sequences  $x[1..m]$  and  $y[1..n]$  and set of transformation-operation costs, the **edit distance** from  $x$  to  $y$  is the cost of the least expensive operation sequence that transforms  $x$  to  $y$ . Describe a dynamic-programming algorithm that finds the edit distance from  $x[1..m]$  to  $y[1..n]$  and prints an optimal operation sequence. Analyze the running time and space requirements of your algorithm.

The edit-distance problem generalizes the problem of aligning two DNA sequences (see, for example, Setubal and Meidanis [310, Section 3.2]). There are several methods for measuring the similarity of two DNA sequences by aligning them. One such method to align two sequences  $x$  and  $y$  consists of inserting spaces at



arbitrary locations in the two sequences (including at either end) so that the resulting sequences  $x'$  and  $y'$  have the same length but do not have a space in the same position (i.e., for no position  $j$  are both  $x'[j]$  and  $y'[j]$  a space). Then we assign a “score” to each position. Position  $j$  receives a score as follows:

- $+1$  if  $x'[j] = y'[j]$  and neither is a space,
- $-1$  if  $x'[j] \neq y'[j]$  and neither is a space,
- $-2$  if either  $x'[j]$  or  $y'[j]$  is a space.

The score for the alignment is the sum of the scores of the individual positions. For example, given the sequences  $x = \text{GATCGGCAT}$  and  $y = \text{CAATGTGAATC}$ , one alignment is

```
G  ATCG  GCAT
CAAT GTGAATC
- * + + + + + - + + *
```

A  $+$  under a position indicates a score of  $+1$  for that position, a  $-$  indicates a score of  $-1$ , and a  $*$  indicates a score of  $-2$ , so that this alignment has a total score of  $6 \cdot 1 - 2 \cdot 1 - 4 \cdot 2 = -4$ .

- b.** Explain how to cast the problem of finding an optimal alignment as an edit distance problem using a subset of the transformation operations copy, replace, delete, insert, twiddle, and kill.

### 15-6 Planning a company party

Professor Stewart is consulting for the president of a corporation that is planning a company party. The company has a hierarchical structure; that is, the supervisor relation forms a tree rooted at the president. The personnel office has ranked each employee with a conviviality rating, which is a real number. In order to make the party fun for all attendees, the president does not want both an employee and his or her immediate supervisor to attend.

Professor Stewart is given the tree that describes the structure of the corporation, using the left-child, right-sibling representation described in Section 10.4. Each node of the tree holds, in addition to the pointers, the name of an employee and that employee’s conviviality ranking. Describe an algorithm to make up a guest list that maximizes the sum of the conviviality ratings of the guests. Analyze the running time of your algorithm.

### 15-7 Viterbi algorithm

We can use dynamic programming on a directed graph  $G = (V, E)$  for speech recognition. Each edge  $(u, v) \in E$  is labeled with a sound  $\sigma(u, v)$  from a finite set  $\Sigma$  of sounds. The labeled graph is a formal model of a person speaking

a restricted language. Each path in the graph starting from a distinguished vertex  $v_0 \in V$  corresponds to a possible sequence of sounds produced by the model. We define the label of a directed path to be the concatenation of the labels of the edges on that path.

- a. Describe an efficient algorithm that, given an edge-labeled graph  $G$  with distinguished vertex  $v_0$  and a sequence  $s = \langle \sigma_1, \sigma_2, \dots, \sigma_k \rangle$  of sounds from  $\Sigma$ , returns a path in  $G$  that begins at  $v_0$  and has  $s$  as its label, if any such path exists. Otherwise, the algorithm should return NO-SUCH-PATH. Analyze the running time of your algorithm. (*Hint:* You may find concepts from Chapter 22 useful.)

Now, suppose that every edge  $(u, v) \in E$  has an associated nonnegative probability  $p(u, v)$  of traversing the edge  $(u, v)$  from vertex  $u$  and thus producing the corresponding sound. The sum of the probabilities of the edges leaving any vertex equals 1. The probability of a path is defined to be the product of the probabilities of its edges. We can view the probability of a path beginning at  $v_0$  as the probability that a “random walk” beginning at  $v_0$  will follow the specified path, where we randomly choose which edge to take leaving a vertex  $u$  according to the probabilities of the available edges leaving  $u$ .

- b. Extend your answer to part (a) so that if a path is returned, it is a *most probable path* starting at  $v_0$  and having label  $s$ . Analyze the running time of your algorithm.

### 15-8 Image compression by seam carving

We are given a color picture consisting of an  $m \times n$  array  $A[1..m, 1..n]$  of pixels, where each pixel specifies a triple of red, green, and blue (RGB) intensities. Suppose that we wish to compress this picture slightly. Specifically, we wish to remove one pixel from each of the  $m$  rows, so that the whole picture becomes one pixel narrower. To avoid disturbing visual effects, however, we require that the pixels removed in two adjacent rows be in the same or adjacent columns; the pixels removed form a “seam” from the top row to the bottom row where successive pixels in the seam are adjacent vertically or diagonally.

- a. Show that the number of such possible seams grows at least exponentially in  $m$ , assuming that  $n > 1$ .
- b. Suppose now that along with each pixel  $A[i, j]$ , we have calculated a real-valued disruption measure  $d[i, j]$ , indicating how disruptive it would be to remove pixel  $A[i, j]$ . Intuitively, the lower a pixel’s disruption measure, the more similar the pixel is to its neighbors. Suppose further that we define the disruption measure of a seam to be the sum of the disruption measures of its pixels.

Give an algorithm to find a seam with the lowest disruption measure. How efficient is your algorithm?

### 15-9 *Breaking a string*

A certain string-processing language allows a programmer to break a string into two pieces. Because this operation copies the string, it costs  $n$  time units to break a string of  $n$  characters into two pieces. Suppose a programmer wants to break a string into many pieces. The order in which the breaks occur can affect the total amount of time used. For example, suppose that the programmer wants to break a 20-character string after characters 2, 8, and 10 (numbering the characters in ascending order from the left-hand end, starting from 1). If she programs the breaks to occur in left-to-right order, then the first break costs 20 time units, the second break costs 18 time units (breaking the string from characters 3 to 20 at character 8), and the third break costs 12 time units, totaling 50 time units. If she programs the breaks to occur in right-to-left order, however, then the first break costs 20 time units, the second break costs 10 time units, and the third break costs 8 time units, totaling 38 time units. In yet another order, she could break first at 8 (costing 20), then break the left piece at 2 (costing 8), and finally the right piece at 10 (costing 12), for a total cost of 40.

Design an algorithm that, given the numbers of characters after which to break, determines a least-cost way to sequence those breaks. More formally, given a string  $S$  with  $n$  characters and an array  $L[1..m]$  containing the break points, compute the lowest cost for a sequence of breaks, along with a sequence of breaks that achieves this cost.

### 15-10 *Planning an investment strategy*

Your knowledge of algorithms helps you obtain an exciting job with the Acme Computer Company, along with a \$10,000 signing bonus. You decide to invest this money with the goal of maximizing your return at the end of 10 years. You decide to use the Amalgamated Investment Company to manage your investments. Amalgamated Investments requires you to observe the following rules. It offers  $n$  different investments, numbered 1 through  $n$ . In each year  $j$ , investment  $i$  provides a return rate of  $r_{ij}$ . In other words, if you invest  $d$  dollars in investment  $i$  in year  $j$ , then at the end of year  $j$ , you have  $dr_{ij}$  dollars. The return rates are guaranteed, that is, you are given all the return rates for the next 10 years for each investment. You make investment decisions only once per year. At the end of each year, you can leave the money made in the previous year in the same investments, or you can shift money to other investments, by either shifting money between existing investments or moving money to a new investment. If you do not move your money between two consecutive years, you pay a fee of  $f_1$  dollars, whereas if you switch your money, you pay a fee of  $f_2$  dollars, where  $f_2 > f_1$ .

- a. The problem, as stated, allows you to invest your money in multiple investments in each year. Prove that there exists an optimal investment strategy that, in each year, puts all the money into a single investment. (Recall that an optimal investment strategy maximizes the amount of money after 10 years and is not concerned with any other objectives, such as minimizing risk.)
- b. Prove that the problem of planning your optimal investment strategy exhibits optimal substructure.
- c. Design an algorithm that plans your optimal investment strategy. What is the running time of your algorithm?
- d. Suppose that Amalgamated Investments imposed the additional restriction that, at any point, you can have no more than \$15,000 in any one investment. Show that the problem of maximizing your income at the end of 10 years no longer exhibits optimal substructure.

### 15-11 *Inventory planning*

The Rinky Dink Company makes machines that resurface ice rinks. The demand for such products varies from month to month, and so the company needs to develop a strategy to plan its manufacturing given the fluctuating, but predictable, demand. The company wishes to design a plan for the next  $n$  months. For each month  $i$ , the company knows the demand  $d_i$ , that is, the number of machines that it will sell. Let  $D = \sum_{i=1}^n d_i$  be the total demand over the next  $n$  months. The company keeps a full-time staff who provide labor to manufacture up to  $m$  machines per month. If the company needs to make more than  $m$  machines in a given month, it can hire additional, part-time labor, at a cost that works out to  $c$  dollars per machine. Furthermore, if, at the end of a month, the company is holding any unsold machines, it must pay inventory costs. The cost for holding  $j$  machines is given as a function  $h(j)$  for  $j = 1, 2, \dots, D$ , where  $h(j) \geq 0$  for  $1 \leq j \leq D$  and  $h(j) \leq h(j + 1)$  for  $1 \leq j \leq D - 1$ .

Give an algorithm that calculates a plan for the company that minimizes its costs while fulfilling all the demand. The running time should be polynomial in  $n$  and  $D$ .

### 15-12 *Signing free-agent baseball players*

Suppose that you are the general manager for a major-league baseball team. During the off-season, you need to sign some free-agent players for your team. The team owner has given you a budget of  $\$X$  to spend on free agents. You are allowed to spend less than  $\$X$  altogether, but the owner will fire you if you spend any more than  $\$X$ .

You are considering  $N$  different positions, and for each position,  $P$  free-agent players who play that position are available.<sup>8</sup> Because you do not want to overload your roster with too many players at any position, for each position you may sign at most one free agent who plays that position. (If you do not sign any players at a particular position, then you plan to stick with the players you already have at that position.)

To determine how valuable a player is going to be, you decide to use a sabermetric statistic<sup>9</sup> known as “VORP,” or “value over replacement player.” A player with a higher VORP is more valuable than a player with a lower VORP. A player with a higher VORP is not necessarily more expensive to sign than a player with a lower VORP, because factors other than a player’s value determine how much it costs to sign him.

For each available free-agent player, you have three pieces of information:

- the player’s position,
- the amount of money it will cost to sign the player, and
- the player’s VORP.

Devise an algorithm that maximizes the total VORP of the players you sign while spending no more than  $\$X$  altogether. You may assume that each player signs for a multiple of \$100,000. Your algorithm should output the total VORP of the players you sign, the total amount of money you spend, and a list of which players you sign. Analyze the running time and space requirement of your algorithm.

---

## Chapter notes

R. Bellman began the systematic study of dynamic programming in 1955. The word “programming,” both here and in linear programming, refers to using a tabular solution method. Although optimization techniques incorporating elements of dynamic programming were known earlier, Bellman provided the area with a solid mathematical basis [37].

---

<sup>8</sup>Although there are nine positions on a baseball team,  $N$  is not necessarily equal to 9 because some general managers have particular ways of thinking about positions. For example, a general manager might consider right-handed pitchers and left-handed pitchers to be separate “positions,” as well as starting pitchers, long relief pitchers (relief pitchers who can pitch several innings), and short relief pitchers (relief pitchers who normally pitch at most only one inning).

<sup>9</sup>*Sabermetrics* is the application of statistical analysis to baseball records. It provides several ways to compare the relative values of individual players.

Galil and Park [125] classify dynamic-programming algorithms according to the size of the table and the number of other table entries each entry depends on. They call a dynamic-programming algorithm  $tD/eD$  if its table size is  $O(n^t)$  and each entry depends on  $O(n^e)$  other entries. For example, the matrix-chain multiplication algorithm in Section 15.2 would be  $2D/1D$ , and the longest-common-subsequence algorithm in Section 15.4 would be  $2D/0D$ .

Hu and Shing [182, 183] give an  $O(n \lg n)$ -time algorithm for the matrix-chain multiplication problem.

The  $O(mn)$ -time algorithm for the longest-common-subsequence problem appears to be a folk algorithm. Knuth [70] posed the question of whether subquadratic algorithms for the LCS problem exist. Masek and Paterson [244] answered this question in the affirmative by giving an algorithm that runs in  $O(mn/\lg n)$  time, where  $n \leq m$  and the sequences are drawn from a set of bounded size. For the special case in which no element appears more than once in an input sequence, Szymanski [326] shows how to solve the problem in  $O((n + m) \lg(n + m))$  time. Many of these results extend to the problem of computing string edit distances (Problem 15-5).

An early paper on variable-length binary encodings by Gilbert and Moore [133] had applications to constructing optimal binary search trees for the case in which all probabilities  $p_i$  are 0; this paper contains an  $O(n^3)$ -time algorithm. Aho, Hopcroft, and Ullman [5] present the algorithm from Section 15.5. Exercise 15.5-4 is due to Knuth [212]. Hu and Tucker [184] devised an algorithm for the case in which all probabilities  $p_i$  are 0 that uses  $O(n^2)$  time and  $O(n)$  space; subsequently, Knuth [211] reduced the time to  $O(n \lg n)$ .

Problem 15-8 is due to Avidan and Shamir [27], who have posted on the Web a wonderful video illustrating this image-compression technique.

---

## 16 Greedy Algorithms

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step. For many optimization problems, using dynamic programming to determine the best choices is overkill; simpler, more efficient algorithms will do. A *greedy algorithm* always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution. This chapter explores optimization problems for which greedy algorithms provide optimal solutions. Before reading this chapter, you should read about dynamic programming in Chapter 15, particularly Section 15.3.

Greedy algorithms do not always yield optimal solutions, but for many problems they do. We shall first examine, in Section 16.1, a simple but nontrivial problem, the activity-selection problem, for which a greedy algorithm efficiently computes an optimal solution. We shall arrive at the greedy algorithm by first considering a dynamic-programming approach and then showing that we can always make greedy choices to arrive at an optimal solution. Section 16.2 reviews the basic elements of the greedy approach, giving a direct approach for proving greedy algorithms correct. Section 16.3 presents an important application of greedy techniques: designing data-compression (Huffman) codes. In Section 16.4, we investigate some of the theory underlying combinatorial structures called “matroids,” for which a greedy algorithm always produces an optimal solution. Finally, Section 16.5 applies matroids to solve a problem of scheduling unit-time tasks with deadlines and penalties.

The greedy method is quite powerful and works well for a wide range of problems. Later chapters will present many algorithms that we can view as applications of the greedy method, including minimum-spanning-tree algorithms (Chapter 23), Dijkstra’s algorithm for shortest paths from a single source (Chapter 24), and Chvátal’s greedy set-covering heuristic (Chapter 35). Minimum-spanning-tree algorithms furnish a classic example of the greedy method. Although you can read

this chapter and Chapter 23 independently of each other, you might find it useful to read them together.

## 16.1 An activity-selection problem

Our first example is the problem of scheduling several competing activities that require exclusive use of a common resource, with a goal of selecting a maximum-size set of mutually compatible activities. Suppose we have a set  $S = \{a_1, a_2, \dots, a_n\}$  of  $n$  proposed **activities** that wish to use a resource, such as a lecture hall, which can serve only one activity at a time. Each activity  $a_i$  has a **start time**  $s_i$  and a **finish time**  $f_i$ , where  $0 \leq s_i < f_i < \infty$ . If selected, activity  $a_i$  takes place during the half-open time interval  $[s_i, f_i)$ . Activities  $a_i$  and  $a_j$  are **compatible** if the intervals  $[s_i, f_i)$  and  $[s_j, f_j)$  do not overlap. That is,  $a_i$  and  $a_j$  are compatible if  $s_i \geq f_j$  or  $s_j \geq f_i$ . In the **activity-selection problem**, we wish to select a maximum-size subset of mutually compatible activities. We assume that the activities are sorted in monotonically increasing order of finish time:

$$f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n. \quad (16.1)$$

(We shall see later the advantage that this assumption provides.) For example, consider the following set  $S$  of activities:

| $i$   | 1 | 2 | 3 | 4 | 5 | 6 | 7  | 8  | 9  | 10 | 11 |
|-------|---|---|---|---|---|---|----|----|----|----|----|
| $s_i$ | 1 | 3 | 0 | 5 | 3 | 5 | 6  | 8  | 8  | 2  | 12 |
| $f_i$ | 4 | 5 | 6 | 7 | 9 | 9 | 10 | 11 | 12 | 14 | 16 |

For this example, the subset  $\{a_3, a_9, a_{11}\}$  consists of mutually compatible activities. It is not a maximum subset, however, since the subset  $\{a_1, a_4, a_8, a_{11}\}$  is larger. In fact,  $\{a_1, a_4, a_8, a_{11}\}$  is a largest subset of mutually compatible activities; another largest subset is  $\{a_2, a_4, a_9, a_{11}\}$ .

We shall solve this problem in several steps. We start by thinking about a dynamic-programming solution, in which we consider several choices when determining which subproblems to use in an optimal solution. We shall then observe that we need to consider only one choice—the greedy choice—and that when we make the greedy choice, only one subproblem remains. Based on these observations, we shall develop a recursive greedy algorithm to solve the activity-scheduling problem. We shall complete the process of developing a greedy solution by converting the recursive algorithm to an iterative one. Although the steps we shall go through in this section are slightly more involved than is typical when developing a greedy algorithm, they illustrate the relationship between greedy algorithms and dynamic programming.



### The optimal substructure of the activity-selection problem

We can easily verify that the activity-selection problem exhibits optimal substructure. Let us denote by  $S_{ij}$  the set of activities that start after activity  $a_i$  finishes and that finish before activity  $a_j$  starts. Suppose that we wish to find a maximum set of mutually compatible activities in  $S_{ij}$ , and suppose further that such a maximum set is  $A_{ij}$ , which includes some activity  $a_k$ . By including  $a_k$  in an optimal solution, we are left with two subproblems: finding mutually compatible activities in the set  $S_{ik}$  (activities that start after activity  $a_i$  finishes and that finish before activity  $a_k$  starts) and finding mutually compatible activities in the set  $S_{kj}$  (activities that start after activity  $a_k$  finishes and that finish before activity  $a_j$  starts). Let  $A_{ik} = A_{ij} \cap S_{ik}$  and  $A_{kj} = A_{ij} \cap S_{kj}$ , so that  $A_{ik}$  contains the activities in  $A_{ij}$  that finish before  $a_k$  starts and  $A_{kj}$  contains the activities in  $A_{ij}$  that start after  $a_k$  finishes. Thus, we have  $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ , and so the maximum-size set  $A_{ij}$  of mutually compatible activities in  $S_{ij}$  consists of  $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$  activities.

The usual cut-and-paste argument shows that the optimal solution  $A_{ij}$  must also include optimal solutions to the two subproblems for  $S_{ik}$  and  $S_{kj}$ . If we could find a set  $A'_{kj}$  of mutually compatible activities in  $S_{kj}$  where  $|A'_{kj}| > |A_{kj}|$ , then we could use  $A'_{kj}$ , rather than  $A_{kj}$ , in a solution to the subproblem for  $S_{ij}$ . We would have constructed a set of  $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$  mutually compatible activities, which contradicts the assumption that  $A_{ij}$  is an optimal solution. A symmetric argument applies to the activities in  $S_{ik}$ .

This way of characterizing optimal substructure suggests that we might solve the activity-selection problem by dynamic programming. If we denote the size of an optimal solution for the set  $S_{ij}$  by  $c[i, j]$ , then we would have the recurrence

$$c[i, j] = c[i, k] + c[k, j] + 1.$$

Of course, if we did not know that an optimal solution for the set  $S_{ij}$  includes activity  $a_k$ , we would have to examine all activities in  $S_{ij}$  to find which one to choose, so that

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset, \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset. \end{cases} \quad (16.2)$$

We could then develop a recursive algorithm and memoize it, or we could work bottom-up and fill in table entries as we go along. But we would be overlooking another important characteristic of the activity-selection problem that we can use to great advantage.

### Making the greedy choice

What if we could choose an activity to add to our optimal solution without having to first solve all the subproblems? That could save us from having to consider all the choices inherent in recurrence (16.2). In fact, for the activity-selection problem, we need consider only one choice: the greedy choice.

What do we mean by the greedy choice for the activity-selection problem? Intuition suggests that we should choose an activity that leaves the resource available for as many other activities as possible. Now, of the activities we end up choosing, one of them must be the first one to finish. Our intuition tells us, therefore, to choose the activity in  $S$  with the earliest finish time, since that would leave the resource available for as many of the activities that follow it as possible. (If more than one activity in  $S$  has the earliest finish time, then we can choose any such activity.) In other words, since the activities are sorted in monotonically increasing order by finish time, the greedy choice is activity  $a_1$ . Choosing the first activity to finish is not the only way to think of making a greedy choice for this problem; Exercise 16.1-3 asks you to explore other possibilities.

If we make the greedy choice, we have only one remaining subproblem to solve: finding activities that start after  $a_1$  finishes. Why don't we have to consider activities that finish before  $a_1$  starts? We have that  $s_1 < f_1$ , and  $f_1$  is the earliest finish time of any activity, and therefore no activity can have a finish time less than or equal to  $s_1$ . Thus, all activities that are compatible with activity  $a_1$  must start after  $a_1$  finishes.

Furthermore, we have already established that the activity-selection problem exhibits optimal substructure. Let  $S_k = \{a_i \in S : s_i \geq f_k\}$  be the set of activities that start after activity  $a_k$  finishes. If we make the greedy choice of activity  $a_1$ , then  $S_1$  remains as the only subproblem to solve.<sup>1</sup> Optimal substructure tells us that if  $a_1$  is in the optimal solution, then an optimal solution to the original problem consists of activity  $a_1$  and all the activities in an optimal solution to the subproblem  $S_1$ .

One big question remains: is our intuition correct? Is the greedy choice—in which we choose the first activity to finish—always part of some optimal solution? The following theorem shows that it is.

---

<sup>1</sup>We sometimes refer to the sets  $S_k$  as subproblems rather than as just sets of activities. It will always be clear from the context whether we are referring to  $S_k$  as a set of activities or as a subproblem whose input is that set.

**Theorem 16.1**

Consider any nonempty subproblem  $S_k$ , and let  $a_m$  be an activity in  $S_k$  with the earliest finish time. Then  $a_m$  is included in some maximum-size subset of mutually compatible activities of  $S_k$ .

**Proof** Let  $A_k$  be a maximum-size subset of mutually compatible activities in  $S_k$ , and let  $a_j$  be the activity in  $A_k$  with the earliest finish time. If  $a_j = a_m$ , we are done, since we have shown that  $a_m$  is in some maximum-size subset of mutually compatible activities of  $S_k$ . If  $a_j \neq a_m$ , let the set  $A'_k = A_k - \{a_j\} \cup \{a_m\}$  be  $A_k$  but substituting  $a_m$  for  $a_j$ . The activities in  $A'_k$  are disjoint, which follows because the activities in  $A_k$  are disjoint,  $a_j$  is the first activity in  $A_k$  to finish, and  $f_m \leq f_j$ . Since  $|A'_k| = |A_k|$ , we conclude that  $A'_k$  is a maximum-size subset of mutually compatible activities of  $S_k$ , and it includes  $a_m$ . ■

Thus, we see that although we might be able to solve the activity-selection problem with dynamic programming, we don't need to. (Besides, we have not yet examined whether the activity-selection problem even has overlapping subproblems.) Instead, we can repeatedly choose the activity that finishes first, keep only the activities compatible with this activity, and repeat until no activities remain. Moreover, because we always choose the activity with the earliest finish time, the finish times of the activities we choose must strictly increase. We can consider each activity just once overall, in monotonically increasing order of finish times.

An algorithm to solve the activity-selection problem does not need to work bottom-up, like a table-based dynamic-programming algorithm. Instead, it can work top-down, choosing an activity to put into the optimal solution and then solving the subproblem of choosing activities from those that are compatible with those already chosen. Greedy algorithms typically have this top-down design: make a choice and then solve a subproblem, rather than the bottom-up technique of solving subproblems before making a choice.

**A recursive greedy algorithm**

Now that we have seen how to bypass the dynamic-programming approach and instead use a top-down, greedy algorithm, we can write a straightforward, recursive procedure to solve the activity-selection problem. The procedure RECURSIVE-ACTIVITY-SELECTOR takes the start and finish times of the activities, represented as arrays  $s$  and  $f$ ,<sup>2</sup> the index  $k$  that defines the subproblem  $S_k$  it is to solve, and

---

<sup>2</sup>Because the pseudocode takes  $s$  and  $f$  as arrays, it indexes into them with square brackets rather than subscripts.

the size  $n$  of the original problem. It returns a maximum-size set of mutually compatible activities in  $S_k$ . We assume that the  $n$  input activities are already ordered by monotonically increasing finish time, according to equation (16.1). If not, we can sort them into this order in  $O(n \lg n)$  time, breaking ties arbitrarily. In order to start, we add the fictitious activity  $a_0$  with  $f_0 = 0$ , so that subproblem  $S_0$  is the entire set of activities  $S$ . The initial call, which solves the entire problem, is  $\text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, 0, n)$ .

$\text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, k, n)$

```

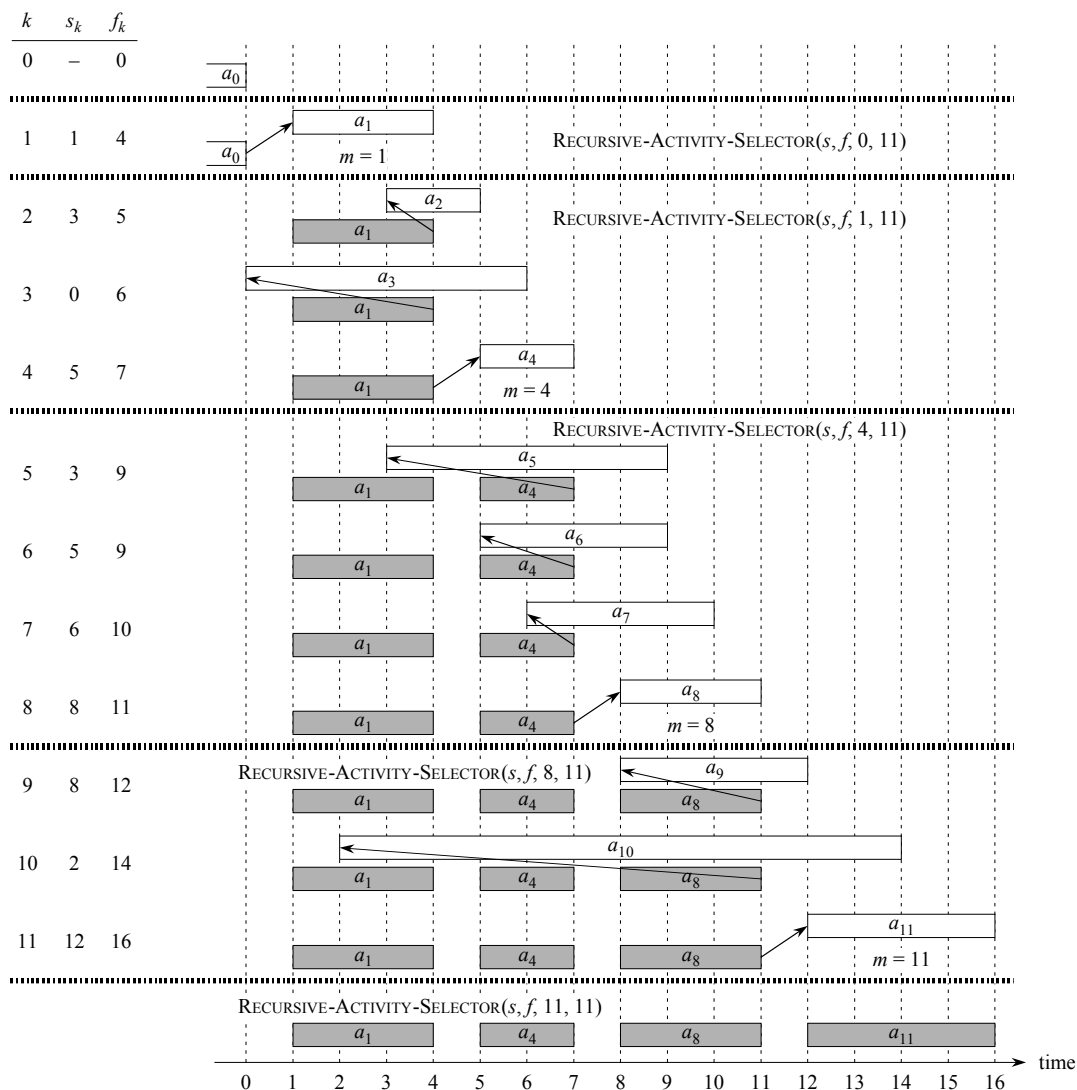
1   $m = k + 1$ 
2  while  $m \leq n$  and  $s[m] < f[k]$       // find the first activity in  $S_k$  to finish
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```

Figure 16.1 shows the operation of the algorithm. In a given recursive call  $\text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, k, n)$ , the **while** loop of lines 2–3 looks for the first activity in  $S_k$  to finish. The loop examines  $a_{k+1}, a_{k+2}, \dots, a_n$ , until it finds the first activity  $a_m$  that is compatible with  $a_k$ ; such an activity has  $s_m \geq f_k$ . If the loop terminates because it finds such an activity, line 5 returns the union of  $\{a_m\}$  and the maximum-size subset of  $S_m$  returned by the recursive call  $\text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ . Alternatively, the loop may terminate because  $m > n$ , in which case we have examined all activities in  $S_k$  without finding one that is compatible with  $a_k$ . In this case,  $S_k = \emptyset$ , and so the procedure returns  $\emptyset$  in line 6.

Assuming that the activities have already been sorted by finish times, the running time of the call  $\text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, 0, n)$  is  $\Theta(n)$ , which we can see as follows. Over all recursive calls, each activity is examined exactly once in the **while** loop test of line 2. In particular, activity  $a_i$  is examined in the last call made in which  $k < i$ .

### An iterative greedy algorithm

We easily can convert our recursive procedure to an iterative one. The procedure  $\text{RECURSIVE-ACTIVITY-SELECTOR}$  is almost “tail recursive” (see Problem 7-4): it ends with a recursive call to itself followed by a union operation. It is usually a straightforward task to transform a tail-recursive procedure to an iterative form; in fact, some compilers for certain programming languages perform this task automatically. As written,  $\text{RECURSIVE-ACTIVITY-SELECTOR}$  works for subproblems  $S_k$ , i.e., subproblems that consist of the last activities to finish.



**Figure 16.1** The operation of RECURSIVE-ACTIVITY-SELECTOR on the 11 activities given earlier. Activities considered in each recursive call appear between horizontal lines. The fictitious activity  $a_0$  finishes at time 0, and the initial call RECURSIVE-ACTIVITY-SELECTOR( $s, f, 0, 11$ ), selects activity  $a_1$ . In each recursive call, the activities that have already been selected are shaded, and the activity shown in white is being considered. If the starting time of an activity occurs before the finish time of the most recently added activity (the arrow between them points left), it is rejected. Otherwise (the arrow points directly up or to the right), it is selected. The last recursive call, RECURSIVE-ACTIVITY-SELECTOR( $s, f, 11, 11$ ), returns  $\emptyset$ . The resulting set of selected activities is  $\{a_1, a_4, a_8, a_{11}\}$ .

The procedure GREEDY-ACTIVITY-SELECTOR is an iterative version of the procedure RECURSIVE-ACTIVITY-SELECTOR. It also assumes that the input activities are ordered by monotonically increasing finish time. It collects selected activities into a set  $A$  and returns this set when it is done.

GREEDY-ACTIVITY-SELECTOR( $s, f$ )

```

1   $n = s.length$ 
2   $A = \{a_1\}$ 
3   $k = 1$ 
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A = A \cup \{a_m\}$ 
7           $k = m$ 
8  return  $A$ 
```

The procedure works as follows. The variable  $k$  indexes the most recent addition to  $A$ , corresponding to the activity  $a_k$  in the recursive version. Since we consider the activities in order of monotonically increasing finish time,  $f_k$  is always the maximum finish time of any activity in  $A$ . That is,

$$f_k = \max \{f_i : a_i \in A\} . \quad (16.3)$$

Lines 2–3 select activity  $a_1$ , initialize  $A$  to contain just this activity, and initialize  $k$  to index this activity. The **for** loop of lines 4–7 finds the earliest activity in  $S_k$  to finish. The loop considers each activity  $a_m$  in turn and adds  $a_m$  to  $A$  if it is compatible with all previously selected activities; such an activity is the earliest in  $S_k$  to finish. To see whether activity  $a_m$  is compatible with every activity currently in  $A$ , it suffices by equation (16.3) to check (in line 5) that its start time  $s_m$  is not earlier than the finish time  $f_k$  of the activity most recently added to  $A$ . If activity  $a_m$  is compatible, then lines 6–7 add activity  $a_m$  to  $A$  and set  $k$  to  $m$ . The set  $A$  returned by the call GREEDY-ACTIVITY-SELECTOR( $s, f$ ) is precisely the set returned by the call RECURSIVE-ACTIVITY-SELECTOR( $s, f, 0, n$ ).

Like the recursive version, GREEDY-ACTIVITY-SELECTOR schedules a set of  $n$  activities in  $\Theta(n)$  time, assuming that the activities were already sorted initially by their finish times.

## Exercises

### 16.1-1

Give a dynamic-programming algorithm for the activity-selection problem, based on recurrence (16.2). Have your algorithm compute the sizes  $c[i, j]$  as defined above and also produce the maximum-size subset of mutually compatible activities.

Assume that the inputs have been sorted as in equation (16.1). Compare the running time of your solution to the running time of GREEDY-ACTIVITY-SELECTOR.

**16.1-2**

Suppose that instead of always selecting the first activity to finish, we instead select the last activity to start that is compatible with all previously selected activities. Describe how this approach is a greedy algorithm, and prove that it yields an optimal solution.

**16.1-3**

Not just any greedy approach to the activity-selection problem produces a maximum-size set of mutually compatible activities. Give an example to show that the approach of selecting the activity of least duration from among those that are compatible with previously selected activities does not work. Do the same for the approaches of always selecting the compatible activity that overlaps the fewest other remaining activities and always selecting the compatible remaining activity with the earliest start time.

**16.1-4**

Suppose that we have a set of activities to schedule among a large number of lecture halls, where any activity can take place in any lecture hall. We wish to schedule all the activities using as few lecture halls as possible. Give an efficient greedy algorithm to determine which activity should use which lecture hall.

(This problem is also known as the *interval-graph coloring problem*. We can create an interval graph whose vertices are the given activities and whose edges connect incompatible activities. The smallest number of colors required to color every vertex so that no two adjacent vertices have the same color corresponds to finding the fewest lecture halls needed to schedule all of the given activities.)

**16.1-5**

Consider a modification to the activity-selection problem in which each activity  $a_i$  has, in addition to a start and finish time, a value  $v_i$ . The objective is no longer to maximize the number of activities scheduled, but instead to maximize the total value of the activities scheduled. That is, we wish to choose a set  $A$  of compatible activities such that  $\sum_{a_k \in A} v_k$  is maximized. Give a polynomial-time algorithm for this problem.

---

## 16.2 Elements of the greedy strategy

A greedy algorithm obtains an optimal solution to a problem by making a sequence of choices. At each decision point, the algorithm makes the choice that seems best at the moment. This heuristic strategy does not always produce an optimal solution, but as we saw in the activity-selection problem, sometimes it does. This section discusses some of the general properties of greedy methods.

The process that we followed in Section 16.1 to develop a greedy algorithm was a bit more involved than is typical. We went through the following steps:

1. Determine the optimal substructure of the problem.
2. Develop a recursive solution. (For the activity-selection problem, we formulated recurrence (16.2), but we bypassed developing a recursive algorithm based on this recurrence.)
3. Show that if we make the greedy choice, then only one subproblem remains.
4. Prove that it is always safe to make the greedy choice. (Steps 3 and 4 can occur in either order.)
5. Develop a recursive algorithm that implements the greedy strategy.
6. Convert the recursive algorithm to an iterative algorithm.

In going through these steps, we saw in great detail the dynamic-programming underpinnings of a greedy algorithm. For example, in the activity-selection problem, we first defined the subproblems  $S_{ij}$ , where both  $i$  and  $j$  varied. We then found that if we always made the greedy choice, we could restrict the subproblems to be of the form  $S_k$ .

Alternatively, we could have fashioned our optimal substructure with a greedy choice in mind, so that the choice leaves just one subproblem to solve. In the activity-selection problem, we could have started by dropping the second subscript and defining subproblems of the form  $S_k$ . Then, we could have proven that a greedy choice (the first activity  $a_m$  to finish in  $S_k$ ), combined with an optimal solution to the remaining set  $S_m$  of compatible activities, yields an optimal solution to  $S_k$ . More generally, we design greedy algorithms according to the following sequence of steps:

1. Cast the optimization problem as one in which we make a choice and are left with one subproblem to solve.
2. Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the greedy choice is always safe.



3. Demonstrate optimal substructure by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.

We shall use this more direct process in later sections of this chapter. Nevertheless, beneath every greedy algorithm, there is almost always a more cumbersome dynamic-programming solution.

How can we tell whether a greedy algorithm will solve a particular optimization problem? No way works all the time, but the greedy-choice property and optimal substructure are the two key ingredients. If we can demonstrate that the problem has these properties, then we are well on the way to developing a greedy algorithm for it.

### Greedy-choice property

The first key ingredient is the *greedy-choice property*: we can assemble a globally optimal solution by making locally optimal (greedy) choices. In other words, when we are considering which choice to make, we make the choice that looks best in the current problem, without considering results from subproblems.

Here is where greedy algorithms differ from dynamic programming. In dynamic programming, we make a choice at each step, but the choice usually depends on the solutions to subproblems. Consequently, we typically solve dynamic-programming problems in a bottom-up manner, progressing from smaller subproblems to larger subproblems. (Alternatively, we can solve them top down, but memoizing. Of course, even though the code works top down, we still must solve the subproblems before making a choice.) In a greedy algorithm, we make whatever choice seems best at the moment and then solve the subproblem that remains. The choice made by a greedy algorithm may depend on choices so far, but it cannot depend on any future choices or on the solutions to subproblems. Thus, unlike dynamic programming, which solves the subproblems before making the first choice, a greedy algorithm makes its first choice before solving any subproblems. A dynamic-programming algorithm proceeds bottom up, whereas a greedy strategy usually progresses in a top-down fashion, making one greedy choice after another, reducing each given problem instance to a smaller one.

Of course, we must prove that a greedy choice at each step yields a globally optimal solution. Typically, as in the case of Theorem 16.1, the proof examines a globally optimal solution to some subproblem. It then shows how to modify the solution to substitute the greedy choice for some other choice, resulting in one similar, but smaller, subproblem.

We can usually make the greedy choice more efficiently than when we have to consider a wider set of choices. For example, in the activity-selection problem, as-

suming that we had already sorted the activities in monotonically increasing order of finish times, we needed to examine each activity just once. By preprocessing the input or by using an appropriate data structure (often a priority queue), we often can make greedy choices quickly, thus yielding an efficient algorithm.

### Optimal substructure

A problem exhibits *optimal substructure* if an optimal solution to the problem contains within it optimal solutions to subproblems. This property is a key ingredient of assessing the applicability of dynamic programming as well as greedy algorithms. As an example of optimal substructure, recall how we demonstrated in Section 16.1 that if an optimal solution to subproblem  $S_{ij}$  includes an activity  $a_k$ , then it must also contain optimal solutions to the subproblems  $S_{ik}$  and  $S_{kj}$ . Given this optimal substructure, we argued that if we knew which activity to use as  $a_k$ , we could construct an optimal solution to  $S_{ij}$  by selecting  $a_k$  along with all activities in optimal solutions to the subproblems  $S_{ik}$  and  $S_{kj}$ . Based on this observation of optimal substructure, we were able to devise the recurrence (16.2) that described the value of an optimal solution.

We usually use a more direct approach regarding optimal substructure when applying it to greedy algorithms. As mentioned above, we have the luxury of assuming that we arrived at a subproblem by having made the greedy choice in the original problem. All we really need to do is argue that an optimal solution to the subproblem, combined with the greedy choice already made, yields an optimal solution to the original problem. This scheme implicitly uses induction on the subproblems to prove that making the greedy choice at every step produces an optimal solution.

### Greedy versus dynamic programming

Because both the greedy and dynamic-programming strategies exploit optimal substructure, you might be tempted to generate a dynamic-programming solution to a problem when a greedy solution suffices or, conversely, you might mistakenly think that a greedy solution works when in fact a dynamic-programming solution is required. To illustrate the subtleties between the two techniques, let us investigate two variants of a classical optimization problem.

The **0-1 knapsack problem** is the following. A thief robbing a store finds  $n$  items. The  $i$ th item is worth  $v_i$  dollars and weighs  $w_i$  pounds, where  $v_i$  and  $w_i$  are integers. The thief wants to take as valuable a load as possible, but he can carry at most  $W$  pounds in his knapsack, for some integer  $W$ . Which items should he take? (We call this the 0-1 knapsack problem because for each item, the thief must either

take it or leave it behind; he cannot take a fractional amount of an item or take an item more than once.)

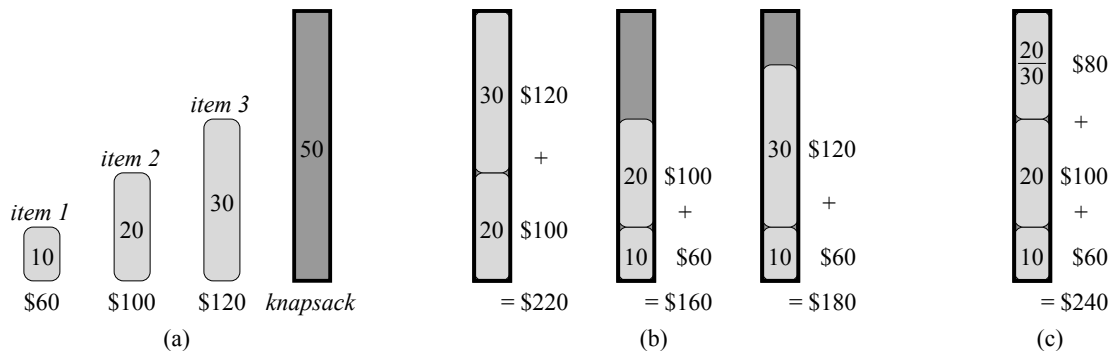
In the **fractional knapsack problem**, the setup is the same, but the thief can take fractions of items, rather than having to make a binary (0-1) choice for each item. You can think of an item in the 0-1 knapsack problem as being like a gold ingot and an item in the fractional knapsack problem as more like gold dust.

Both knapsack problems exhibit the optimal-substructure property. For the 0-1 problem, consider the most valuable load that weighs at most  $W$  pounds. If we remove item  $j$  from this load, the remaining load must be the most valuable load weighing at most  $W - w_j$  that the thief can take from the  $n - 1$  original items excluding  $j$ . For the comparable fractional problem, consider that if we remove a weight  $w$  of one item  $j$  from the optimal load, the remaining load must be the most valuable load weighing at most  $W - w$  that the thief can take from the  $n - 1$  original items plus  $w_j - w$  pounds of item  $j$ .

Although the problems are similar, we can solve the fractional knapsack problem by a greedy strategy, but we cannot solve the 0-1 problem by such a strategy. To solve the fractional problem, we first compute the value per pound  $v_i/w_i$  for each item. Obeying a greedy strategy, the thief begins by taking as much as possible of the item with the greatest value per pound. If the supply of that item is exhausted and he can still carry more, he takes as much as possible of the item with the next greatest value per pound, and so forth, until he reaches his weight limit  $W$ . Thus, by sorting the items by value per pound, the greedy algorithm runs in  $O(n \lg n)$  time. We leave the proof that the fractional knapsack problem has the greedy-choice property as Exercise 16.2-1.

To see that this greedy strategy does not work for the 0-1 knapsack problem, consider the problem instance illustrated in Figure 16.2(a). This example has 3 items and a knapsack that can hold 50 pounds. Item 1 weighs 10 pounds and is worth 60 dollars. Item 2 weighs 20 pounds and is worth 100 dollars. Item 3 weighs 30 pounds and is worth 120 dollars. Thus, the value per pound of item 1 is 6 dollars per pound, which is greater than the value per pound of either item 2 (5 dollars per pound) or item 3 (4 dollars per pound). The greedy strategy, therefore, would take item 1 first. As you can see from the case analysis in Figure 16.2(b), however, the optimal solution takes items 2 and 3, leaving item 1 behind. The two possible solutions that take item 1 are both suboptimal.

For the comparable fractional problem, however, the greedy strategy, which takes item 1 first, does yield an optimal solution, as shown in Figure 16.2(c). Taking item 1 doesn't work in the 0-1 problem because the thief is unable to fill his knapsack to capacity, and the empty space lowers the effective value per pound of his load. In the 0-1 problem, when we consider whether to include an item in the knapsack, we must compare the solution to the subproblem that includes the item with the solution to the subproblem that excludes the item before we can make the



**Figure 16.2** An example showing that the greedy strategy does not work for the 0-1 knapsack problem. **(a)** The thief must select a subset of the three items shown whose weight must not exceed 50 pounds. **(b)** The optimal subset includes items 2 and 3. Any solution with item 1 is suboptimal, even though item 1 has the greatest value per pound. **(c)** For the fractional knapsack problem, taking the items in order of greatest value per pound yields an optimal solution.

choice. The problem formulated in this way gives rise to many overlapping sub-problems—a hallmark of dynamic programming, and indeed, as Exercise 16.2-2 asks you to show, we can use dynamic programming to solve the 0-1 problem.

## Exercises

### 16.2-1

Prove that the fractional knapsack problem has the greedy-choice property.

### 16.2-2

Give a dynamic-programming solution to the 0-1 knapsack problem that runs in  $O(nW)$  time, where  $n$  is the number of items and  $W$  is the maximum weight of items that the thief can put in his knapsack.

### 16.2-3

Suppose that in a 0-1 knapsack problem, the order of the items when sorted by increasing weight is the same as their order when sorted by decreasing value. Give an efficient algorithm to find an optimal solution to this variant of the knapsack problem, and argue that your algorithm is correct.

### 16.2-4

Professor Gekko has always dreamed of inline skating across North Dakota. He plans to cross the state on highway U.S. 2, which runs from Grand Forks, on the eastern border with Minnesota, to Williston, near the western border with Montana.

The professor can carry two liters of water, and he can skate  $m$  miles before running out of water. (Because North Dakota is relatively flat, the professor does not have to worry about drinking water at a greater rate on uphill sections than on flat or downhill sections.) The professor will start in Grand Forks with two full liters of water. His official North Dakota state map shows all the places along U.S. 2 at which he can refill his water and the distances between these locations.

The professor's goal is to minimize the number of water stops along his route across the state. Give an efficient method by which he can determine which water stops he should make. Prove that your strategy yields an optimal solution, and give its running time.

### 16.2-5

Describe an efficient algorithm that, given a set  $\{x_1, x_2, \dots, x_n\}$  of points on the real line, determines the smallest set of unit-length closed intervals that contains all of the given points. Argue that your algorithm is correct.

### 16.2-6 ★

Show how to solve the fractional knapsack problem in  $O(n)$  time.

### 16.2-7

Suppose you are given two sets  $A$  and  $B$ , each containing  $n$  positive integers. You can choose to reorder each set however you like. After reordering, let  $a_i$  be the  $i$ th element of set  $A$ , and let  $b_i$  be the  $i$ th element of set  $B$ . You then receive a payoff of  $\prod_{i=1}^n a_i^{b_i}$ . Give an algorithm that will maximize your payoff. Prove that your algorithm maximizes the payoff, and state its running time.

---

## 16.3 Huffman codes

Huffman codes compress data very effectively: savings of 20% to 90% are typical, depending on the characteristics of the data being compressed. We consider the data to be a sequence of characters. Huffman's greedy algorithm uses a table giving how often each character occurs (i.e., its frequency) to build up an optimal way of representing each character as a binary string.

Suppose we have a 100,000-character data file that we wish to store compactly. We observe that the characters in the file occur with the frequencies given by Figure 16.3. That is, only 6 different characters appear, and the character *a* occurs 45,000 times.

We have many options for how to represent such a file of information. Here, we consider the problem of designing a **binary character code** (or **code** for short)

|                          | a   | b   | c   | d   | e    | f    |
|--------------------------|-----|-----|-----|-----|------|------|
| Frequency (in thousands) | 45  | 13  | 12  | 16  | 9    | 5    |
| Fixed-length codeword    | 000 | 001 | 010 | 011 | 100  | 101  |
| Variable-length codeword | 0   | 101 | 100 | 111 | 1101 | 1100 |

**Figure 16.3** A character-coding problem. A data file of 100,000 characters contains only the characters a–f, with the frequencies indicated. If we assign each character a 3-bit codeword, we can encode the file in 300,000 bits. Using the variable-length code shown, we can encode the file in only 224,000 bits.

in which each character is represented by a unique binary string, which we call a **codeword**. If we use a **fixed-length code**, we need 3 bits to represent 6 characters: a = 000, b = 001, ..., f = 101. This method requires 300,000 bits to code the entire file. Can we do better?

A **variable-length code** can do considerably better than a fixed-length code, by giving frequent characters short codewords and infrequent characters long codewords. Figure 16.3 shows such a code; here the 1-bit string 0 represents a, and the 4-bit string 1100 represents f. This code requires

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000 = 224,000 \text{ bits}$$

to represent the file, a savings of approximately 25%. In fact, this is an optimal character code for this file, as we shall see.

### Prefix codes

We consider here only codes in which no codeword is also a prefix of some other codeword. Such codes are called **prefix codes**.<sup>3</sup> Although we won't prove it here, a prefix code can always achieve the optimal data compression among any character code, and so we suffer no loss of generality by restricting our attention to prefix codes.

Encoding is always simple for any binary character code; we just concatenate the codewords representing each character of the file. For example, with the variable-length prefix code of Figure 16.3, we code the 3-character file abc as  $0 \cdot 101 \cdot 100 = 0101100$ , where “ $\cdot$ ” denotes concatenation.

Prefix codes are desirable because they simplify decoding. Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous. We can simply identify the initial codeword, translate it back to the original char-

---

<sup>3</sup>Perhaps “prefix-free codes” would be a better name, but the term “prefix codes” is standard in the literature.